



Kernel Security: how2rootkit



Capstone: Full-Chain + Rootkit

Academic Integrity and Mental Health

TL;DR:

- I give incompletes.
- I am not a cop.
- I give extensions.
- If you tell me you need an incomplete, I will grant one, no questions asked. You don't need to explain your situation if you're low on emotional energy.
- The capstone is a crucible. Start early, ask for help.
- **Do not work alone.**

HW --> Capstone

- Every homework going forward will build a piece of the capstone.
- Don't underestimate the amount of work involved with integration.

HW	What You Built	Capstone Stage
HW1	AArch64 svc #0 stubs, instruction encoding	Stage 0: remote shellcode
HW2	Tracing syscalls, understanding the call path	Background knowledge
HW3	LD_PRELOAD interception, GOT/PLT patching	Part A: userland rootkit
HW4	kprobes, ftrace, syscall table hooking	Part B: kernel rootkit

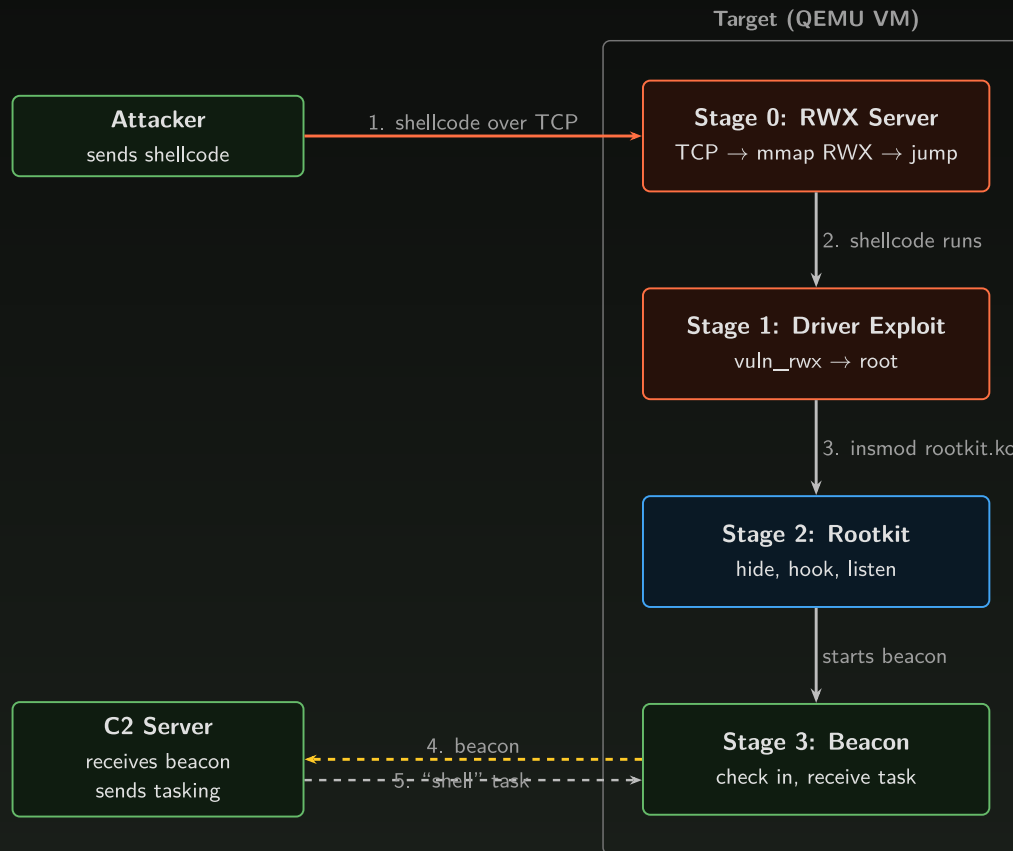
The Scenario

You receive a target machine running a vulnerable server. Your job: own it completely.

- Exploit a vulnerable network service to get a foothold
- Exploits a buggy kernel driver to escalate to root
- Loads your kernel module.
- The rootkit hides files, itself, and opens a network backdoor.
- A beacon phones home to your C2 server. You send tasking. The target executes it.

A defender running `ps`, `lsmod`, `ls`, `netstat` sees nothing unusual. Your operator console shows a compromised host.

Full Chain



Deliverables

- **Shellcode:** AArch64 position-independent code that exploits the target driver and loads the rootkit
- **Driver exploit:** Kernel shellcode via `vuln_rwx.ko` (all students) or RW exploit via `vuln_rw.ko` (grads)
- **rootkit.ko:** Kernel module with file hiding, module hiding, code injection, network backdoor, process hiding
- **librootkit.so:** Userland beacon
- **C2 server:** Receives beacons, displays status, sends tasking (Python)
- **memfd ELF loader:** Run payloads from memory without touching disk
- **Written analysis:** Architecture description, detection strategies, mechanism comparison
- **Special feature:** One per team, instructor-approved

Teams and Logistics

- Teams of 2–3. Private git repo. Add my account so I can track contributions.
- Target: Linux 6.6 on AArch64 QEMU, same environment as HW0–HW4.
- Duration: 6 weeks (March 9 to April 19) with graded milestones.
- **Teams due before spring break (Sunday, March 1).**
- **Special feature proposals due Tuesday, March 10.** Each team must schedule time to meet with me to discuss.
- **Final poster session date TBD.** Open to visitors. Prepare a poster and live demo.
- **Do not work alone.** The workload is designed for a team. A solo attempt will not go well.
- Doing well on this project will **literally** pay off: show off your work to folks who attend the poster session.

Stage 0: The Vulnerable Server

The target runs a TCP server on a configurable port. The server source code is provided

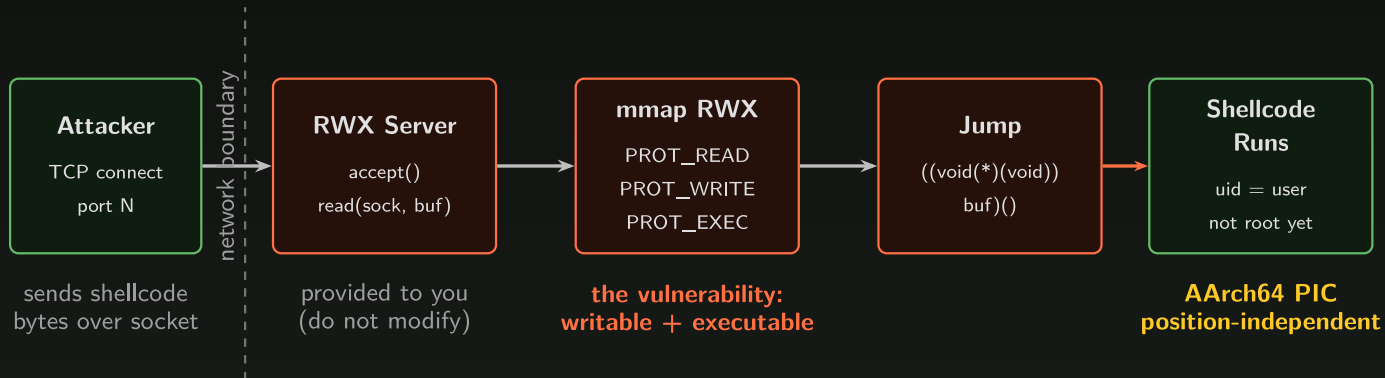
How it works:

1. Accepts a TCP connection
2. You run an "exploit" that results in the vulnerable service mmaping a page with `PROT_READ | PROT_WRITE | PROT_EXEC`
3. Reads up to N bytes from the socket into that page
4. Jumps to byte 0

This is deliberately trivial.

Stage 0

Stage 0: Network Exploitation



Shellcode

- AArch64 position-independent code. Same svc #0 pattern as HW1
- Either delivers a kernel exploit, or bootstraps a second stage (easier)
- Stage one needs to exploit a vulnerable driver. The driver will be released after break.

Exploit as follows:

1. Open the vuln_rwx device (/dev/vuln_rwx)
2. Build kernel shellcode
3. Exploit the vulnerable driver and use it to escalate to root
4. Verify root: `getuid()` should return 0
5. Load the next stage: `execve("/sbin/insmod", ["insmod",
"/path/to/rootkit.ko", NULL], ...)`

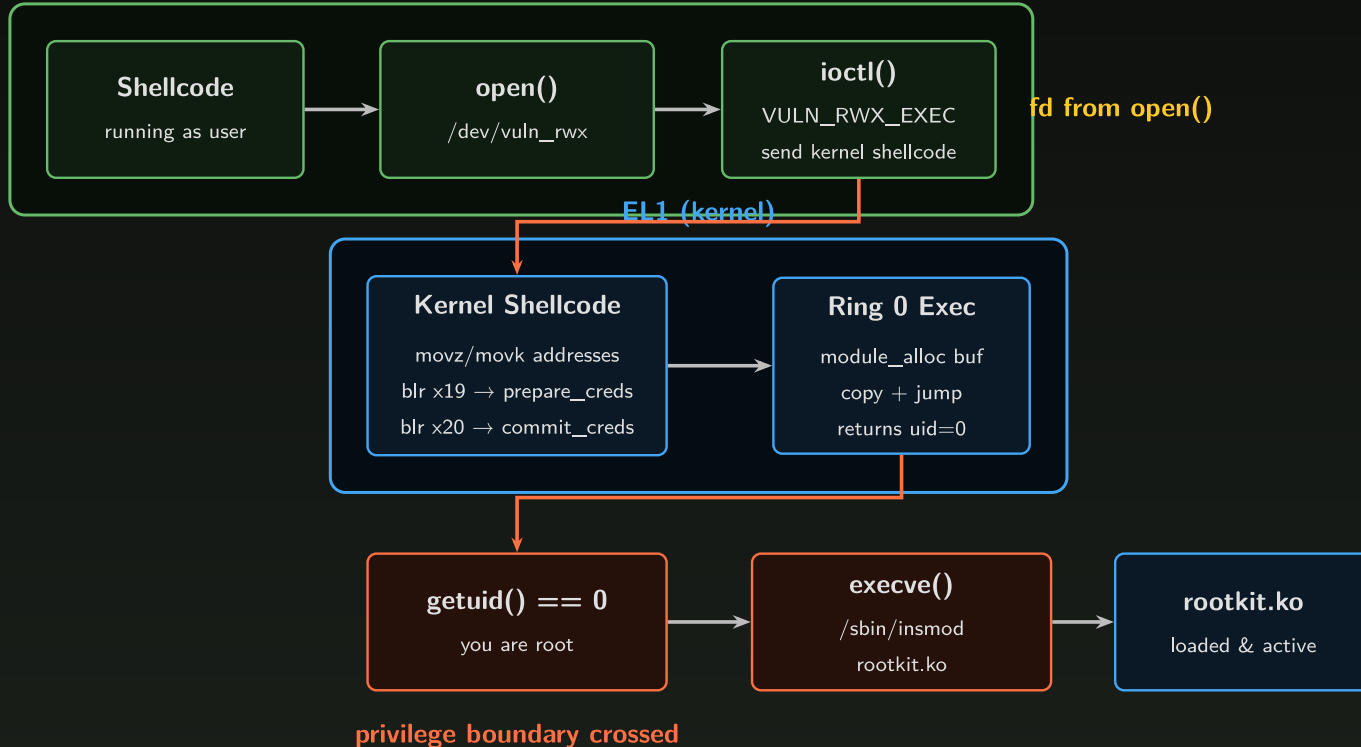
vuln_rwx.ko: All Students

- Buggy JIT engine: `module_alloc()` → copy user code → execute in ring 0.
- Exploit is also trivial. The post exploitation work is a bit more challenging
 - Prizes for code golfing

Stage 1: The Full Path

Stage 1: Driver Exploit → Root

EL0 (user)



vuln_rw.ko: Grad Students

- No code execution primitive, just arbitrary kernel memory read/write.
- Undergrads: bonus points
- Bonus points to grad students who come up with a strategy that isn't just cred overwrite

Grad Path: RW to Code Execution

The `modprobe_path` approach is worth studying even if you use the simpler cred overwrite:

1. `kread()` the address of `modprobe_path` from `/proc/kallsyms`
2. `kwrite()` to overwrite it with `/tmp/rk_load.sh`
3. Your script runs as root when anyone executes an unrecognized binary format
4. The script loads `rootkit.ko` via `modprobe`, indistinguishable from a legitimate driver load
5. Restore original `modprobe_path`

Your process never becomes root. The kernel loads the rootkit for you. This is the same technique used in real LPEs

Loading the Rootkit

After privilege escalation, load `rootkit.ko`. From your shellcode context, you have two options:

Option 1: `execve`:

```
// From userland shellcode (after getuid() == 0)
execve("/sbin/insmod", ["/sbin/insmod", "/path/to/rootkit.ko", NULL], envp);
```

Option 2: `call_usermodehelper` pattern (from kernel shellcode): The kernel can spawn usermode processes via `call_usermodehelper()`. Your kernel shellcode could call this directly. Same pattern as the `modprobe_path` exploit. (Bonus points for undergrad, required for grad)

Part A: Userland Rootkit

LD_PRELOAD library. Same pattern as HW3: `dlsym(RTLD_NEXT, ...)` to intercept libc calls. Four features:

Feature	Mechanism	Points
File/directory hiding	Hook <code>readdir()</code> / <code>readdir64()</code> , skip entries with prefix	15
Process hiding	Filter <code>/proc</code> entries, hide target PIDs	10
memfd ELF loader	Load and execute binaries from memory	10
Self-concealment	Filter <code>/proc/self/maps</code> to hide the library	5

This is HW3 with more targets. The memfd loader is the one new piece.

memfd ELF Loader

`memfd_create()` gives you an anonymous in-memory file. Combined with `fexecve()`, you run a binary without touching disk.

```
int fd = memfd_create("", MFD_CLOEXEC);
write(fd, elf_bytes, elf_len);
fexecve(fd, argv, envp);
// or: snprintf(path, sizeof(path), "/proc/self/fd/%d", fd);
//      execve(path, argv, envp);
```

- C2 server sends an ELF payload over the beacon channel
- Beacon writes it to a memfd
- Beacon executes via `/proc/self/fd/N`
- No file on disk. Invisible to filesystem forensics.

LD_PRELOAD Gotchas

Things that will bite you if you're not careful:

- **readdir vs readdir64**: Different programs call different versions. Hook both or you'll miss half the targets.
- **/proc/self/maps leaks the library path**: If you don't filter it, `cat /proc/self/maps | grep librootkit` reveals you immediately.
- **LD_PRELOAD environment variable**: Visible in `/proc/<pid>/environ`. A paranoid defender will check there too.

Quiz warning: you will be asked why userland rootkits can't hide from static binaries.

Part B: Kernel Rootkit Overview

Five features, three hooking mechanisms. One kernel module.

Feature	Mechanism	Points
File hiding	kretprobe on getdents64	15
Module self-hiding	<code>list_del_init()</code>	5
Code injection	<code>access_process_vm</code> + PC redirect	15
Network backdoor	Hidden TCP listener + magic handshake	20
Process hiding	kretprobe on <code>/proc</code> getdents64	15

File Hiding + Module Self-Hiding

B1: File hiding: kretprobe on `__arm64_sys_getdents64`. Same technique as `cloak.c`:

- Entry handler saves the userspace buffer pointer (double `pt_regs`)
- Return handler copies the `dirent` buffer to kernel space, walks the `linux_dirent64` chain, removes entries with the hidden prefix, copies back
- `ls` sees a clean directory. `cat` still works on hidden files: hiding is not blocking.

B3: Module self-hiding:

Hidden from `lsmod` and `/proc/modules`.

- Special feature: `/sys/module/rootkit/` still exists. Combining with the `sysfs/kobject` hiding removes that too.

Code Injection

Hijack a sleeping userland process to execute attacker-controlled shellcode.

1. Find target: `find_task_by_vpid()` via RCU
2. Verify sleeping: `task->__state != TASK_RUNNING`
3. Find a `VM_EXEC` page in the target's VMAs
4. Write shellcode via `access_process_vm()` with `FOLL_WRITE | FOLL_FORCE`
5. Save original PC: `task_pt_regs(task)->regs[28] = saved_pc`
6. Redirect: `task_pt_regs(task)->pc = shellcode_addr`
7. Target wakes, executes shellcode, returns to original PC via `br x28`

Hard mode: create a new PTEs from scratch for stealth.

`access_process_vm()` into an existing executable page is sufficient for the base requirement. PTE-based injection is bonus points.

Network Backdoor

A hidden TCP listener inside the kernel.

Requirements:

- Listen on a configurable port (compiled-in or set via C2)
- First N bytes of any connection must match a compiled-in magic secret
- Match: parse the rest as a command, execute, send result
- No match: drop the connection silently. No error, no response.
- The port does not show up in netstat or ss by default (combine with network hiding special feature for full stealth)

Kernel socket API: `sock_create()`, `kernel_bind()`, `kernel_listen()`, `kernel_accept()`, `kernel_recvmsg()`, `kernel_sendmsg()`. Run the listener in a kthread.

Process Hiding

- Filter `getdents64` `/proc` directory entries by PID instead of filtering regular file entries by name prefix.
- Maintain list of hidden processes
- include way for hidden processes to see eachother (i.e. magic guid is a good choice)

How `ps` works:

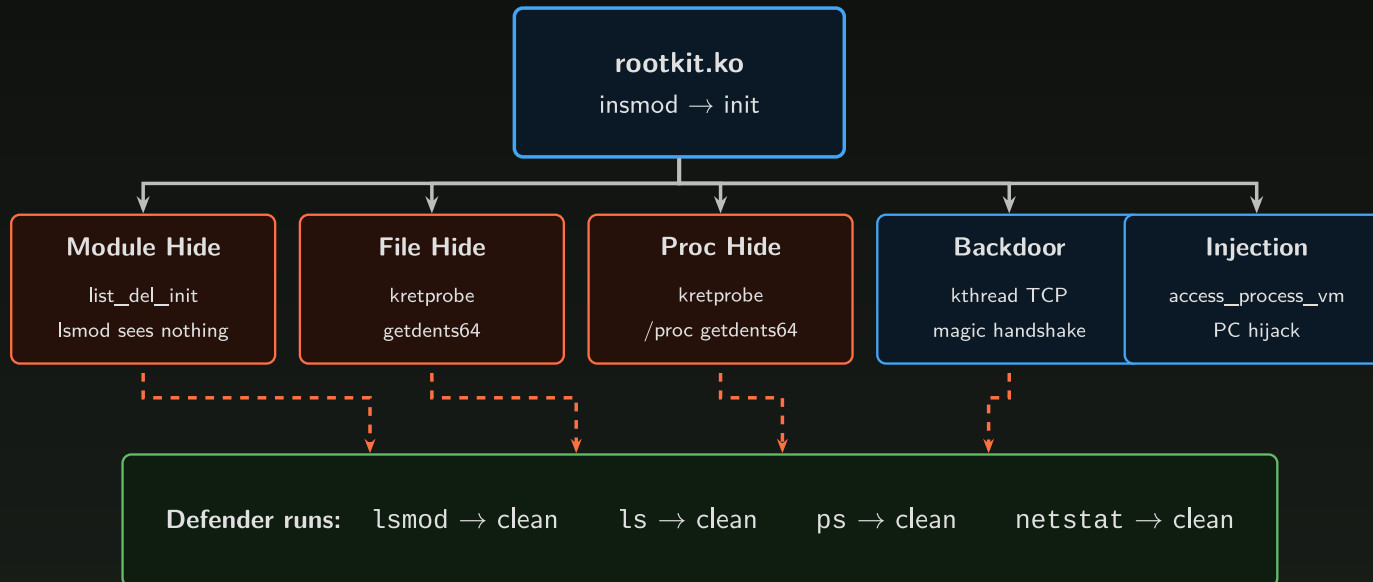
1. `openat(AT_FDCWD, "/proc", ...)` → `fd`
2. `getdents64(fd, buf, count)` → directory entries like `"1"`, `"142"`, `"self"`, ...
3. For each numeric entry, read `/proc/<pid>/stat`, `/proc/<pid>/status`, etc.

Rootkit TLDR

Feature	Status	Effect
File hiding	Active	ls doesn't show rk_* files
Module self-hiding	Active	lsmod doesn't show rootkit
Code injection	Armed	C2 command triggers injection into target PID
Network backdoor	Listening	Hidden port accepts magic-authenticated connections
Process hiding	Active	ps doesn't show hidden PIDs

running rootkit

Stage 2: Rootkit Activates



TCP Beacon

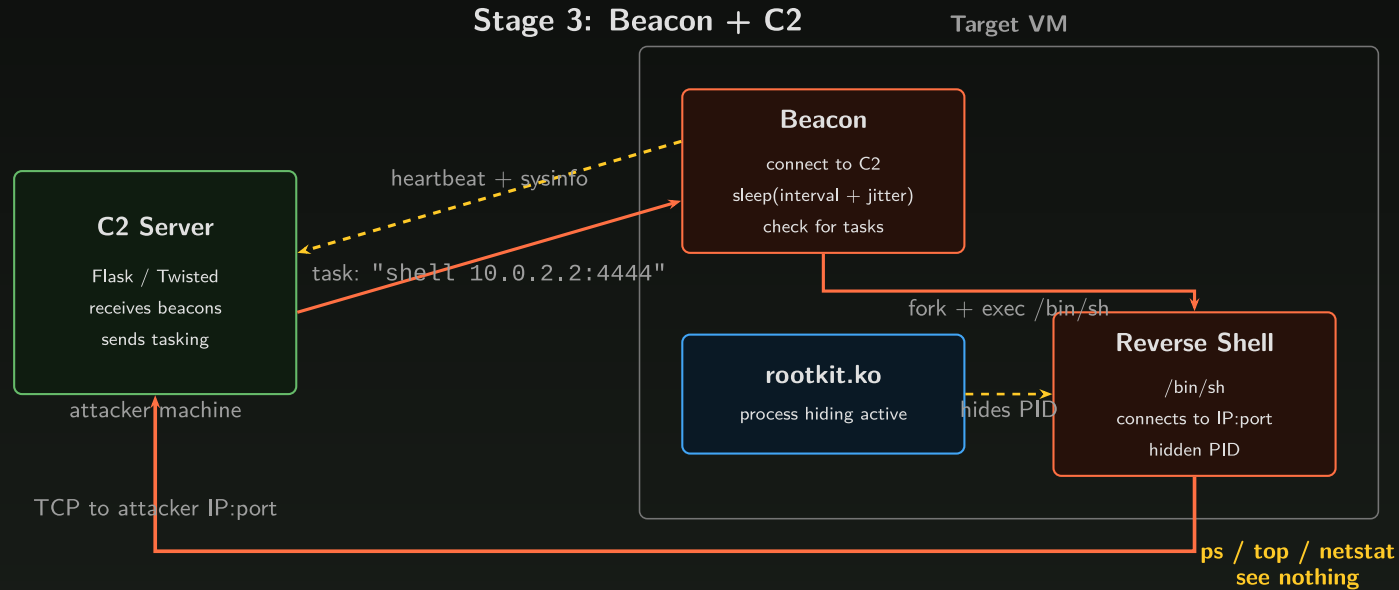
A userland process (loaded via memfd or run directly) that connects back to your C2 server on a timer.

The loop:

1. Connect to C2 address:port (compiled-in)
2. Send heartbeat: hostname, kernel version, uid, PID
3. C2 responds with a task (or "nop": nothing to do)
4. If task is "shell": fork, exec `/bin/sh` with stdin/stdout/stderr redirected to a TCP socket pointing at the operator's IP:port. That's your reverse shell.
5. The rootkit's process hiding covers the shell's PID. ps sees nothing.
6. Sleep for interval + jitter. Go to 1.

That's it. No RPC framework, no structured tasking, no message broker. Connect, check for work, maybe spawn a shell, sleep, repeat.

Beacon + C2 + Reverse Shell



Beacon + C2 + Reverse shell

The beacon phones home. If the C2 says "shell 10.0.2.2:4444", the beacon forks and execs `/bin/sh` connected to that address. The rootkit hides the shell process. The operator gets an interactive shell on a target that looks clean to every standard tool.

The C2 server is equally simple: Python, listen for beacons, display status, type a command when you want a shell. A raw TCP server with a dict is fine. Keep it simple. Note this should be a hidden process

End-to-End Demo

This is what your final demo looks like:

1. Start the vulnerable server on the target QEMU VM
2. From your machine: send shellcode over TCP
3. Shellcode exploits `vuln_rwx.ko` → root
4. Shellcode loads `rootkit.ko`
5. Rootkit activates: files hidden, module hidden, processes hidden, backdoor listening
6. Beacon starts, phones home to C2
7. C2 console shows: new implant connected
8. Operator sends "shell" → reverse shell connects back, hidden from `ps`
9. On the target: `lsmod`, `ps`, `ls`, `netstat`: all clean

This is the demo you will give at the poster session. If any stage breaks, the chain breaks. Test the integration early: by April 12, not the final week.

Special Features

Each team implements a feature beyond the base requirements. Required for the base grade: this is not optional.

Must be approved by instructor. **Proposal due Tuesday, March 10.** Each team must schedule a meeting with me during the week of March 9 to discuss your choice. Show me you went deep on something. There will be lectures given on some special features.

Higher complexity earns more respect (and a better grade). A port knocker is fine. A reflective kernel module loader is impressive. Pick something that interests you and that you can actually finish in the time you have.

Feature Menu: Network and Stealth

- **Port knocker:** SYN sequence to specific ports unlocks the hidden backdoor port
- **Network connection hiding:** Filter `/proc/net/tcp` to hide active connections from `netstat/ss`
- **Kernel log sanitization:** Filter `dmesg` output to remove rootkit-related messages
- **Timestomping:** Fake file timestamps: `stat` shows original times on modified files
- **sysfs/kobject hiding:** Remove `/sys/module/rootkit/`, the one thing `list_del_init` doesn't catch
- **Encrypted beacon payload:** XOR, RC4, or ChaCha20 on C2 traffic
- **DNS covert channel:** Beacon over DNS TXT records instead of raw TCP
- **Anti-module loading:** Hook `finit_module` to block other `insmod` calls after your rootkit loads

Feature Menu: Detection and Advanced

- **Rootkit detector:** Scan for registered kprobes, compare syscall table against `System.map`, diff code pages against on-disk ELF
- **Privilege escalation on demand:**
`prepare_creds()/commit_creds()` triggered via C2 command (reference: `privesc.c`)
- **Alternative hooking:** No `ftrace`, no kprobes: syscall table patching, inline hooks, or eBPF (auto A)
- **Persistent backdoor:** Survive reboots via `systemd` service, cron job, or `initramfs` hook
- **Anti-debug:** Detect `strace/gdb` attachment, kill the debugger or alter behavior
- **Keylogger:** Hook the input subsystem to capture keystrokes
- **Thread injection with PTE mapping:** VMA-less mapping invisible to `/proc/pid/maps` (reference: `threadinject.c`)
- **Container/namespace awareness:** Detect if running inside a container, adapt behavior

Undergrad Rubric

Component	Pts
Part A: Userland Rootkit	40
File/dir hiding (readdir/readdir64)	15
Process hiding (/proc filtering)	10
memfd ELF loader	10
Self-concealment (/proc/self/maps)	5
Part B: Kernel Rootkit	70
File hiding (getdents64 kretprobe)	15
Module self-hiding	5
Code injection (access_process_vm)	15
Network backdoor (hidden port)	20
Process hiding (from kernel)	15

Undergrad Rubric (continued)

Component	Pts
Part C: The Chain	40
Stage 0 shellcode (remote to RWX)	10
Stage 1 exploit (vuln_rwx to root)	15
Stage 2 rootkit load + activate	5
Stage 3 beacon to C2	10
Part D: Analysis	15
Architecture + detection strategies	10
Mechanism comparison (userland vs kernel)	5
Code Quality	10
Special Feature (per student)	15
Demo	10
TOTAL (team of 2)	~215
TOTAL (team of 3)	~230

Graduate Rubric

Same base as undergrad, plus:

Additional Grad Requirement	Pts
RW driver exploitation (vuln_rw, no trivial insmod)	+25
Extended analysis (driver exploit writeup)	+5
Special feature complexity (higher bar)	+5/student
Grad total (team of 2)	~260
Grad total (team of 3)	~280

Grad

Grads must exploit `vu1n_rw.ko` using the read/write primitives. The minimum path is cred overwrite followed by `insmod`. Better approaches (`modprobe_path`, function pointer hijack) earn full marks.

Extra credit (any student): Challenges 1–6 from the spec (+15 each). Challenge 10 super challenge (+40). Challenge 10b reflective kernel module loader (+50). Additional special features beyond the required one (+10 each, instructor approval).

Milestones

Six weeks of development, plus spring break to get started. Plan around these dates.

Date	Milestone
Sun, March 1	Teams formed, private repo created, instructor added
March 2-8	<i>Spring break</i> : start Part A, review your HW3 code
Tue, March 10	Special feature proposals due (one per team member)
Fri, March 13	Instructor meeting completed: schedule during week of March 9
Sun, March 22	Part A complete (LD_PRELOAD features passing test scripts)
Sun, March 29	Part B: file hiding + module hiding + process hiding
Sun, April 5	Part B: code injection + network backdoor; beacon + C2 functional
Sun, April 12	Full chain working; grad RW exploit complete; special features integrated
Sun, April 19	Analysis written, final submission, poster session

Milestones are not suggestions. Missing a milestone costs you time and grades. Your previous homeworks already built most of the pieces: this is integration, not starting from scratch.

Code Quality and Build

- Functions under 50 lines. Files under 500 lines. Break things up.
- make builds everything. One command. No manual steps.
- Error handling with goto unwinding in kernel code. Check every return value.
- No memory leaks. kfree every kmalloc. put_task_struct every get_task_struct.
- No compiler warnings. -Wall -Werror for userland, ccflags-y += -Werror for kernel.
- If it does not build, it does not get graded.
- Make my life easy: conf.h for configuration, make all to build, make test to verify.

Poster Session and Demo (TBD)

Live demo on AArch64 QEMU at the poster session. Show the following:

1. The full chain end-to-end: shellcode lands, driver exploited, rootkit loads, beacon checks in
2. Each rootkit feature individually: file hiding, module hiding, process hiding, code injection, network backdoor
3. C2 server receiving beacon, displaying status, sending a command, showing the result
4. Special feature(s) in action
5. Answer questions from other teams and visitors about your design choices

Prepare a poster. The session is open to visitors.

The test scripts in `test/` match what I run during grading. If they pass, the demo will go fine. Record a backup video in case of live demo failures.

Title

- foo bar



